

Rust Iterator Adapters (Wrappers)

An iterator adapter consumes one `Iterator` and returns a **new `Iterator`**. All adapters are **lazy** — nothing executes until a consuming adapter is called (`.collect()`, `.sum()`, `.for_each()`, `.count()`, `.fold()`, etc.).

Complete Table of Stable Iterator Adapters

Transform

Adapter	Wrapper Type	Description
<code>.map(f)</code>	<code>Map<I, F></code>	Transform each element via closure <code>f</code>
<code>.filter(p)</code>	<code>Filter<I, F></code>	Keep elements where predicate <code>p</code> returns <code>true</code>
<code>.filter_map(f)</code>	<code>FilterMap<I, F></code>	Like <code>.map</code> but discards <code>None</code> results (one pass)
<code>.flat_map(f)</code>	<code>FlatMap<I, F></code>	Map to iterators, then flatten
<code>.flatten()</code>	<code>Flatten<I></code>	Flatten one level of nesting (<code>I: IntoIterator<Item=IntoIterator></code>)
<code>.map_while(p)</code>	<code>MapWhile<I, F></code>	Map elements while predicate returns <code>Some</code> , stops on <code>None</code>

Skip / Take

Adapter	Wrapper Type	Description
<code>.take(n)</code>	<code>Take<I></code>	Yield at most <code>n</code> elements
<code>.skip(n)</code>	<code>Skip<I></code>	Skip first <code>n</code> elements
<code>.take_while(p)</code>	<code>TakeWhile<I, F></code>	Yield while predicate <code>p</code> is true; stops on first <code>false</code>
<code>.skip_while(p)</code>	<code>SkipWhile<I, F></code>	Skip while predicate <code>p</code> is true; yields rest afterwards
<code>.step_by(n)</code>	<code>StepBy<I></code>	Yield every <code>n</code> -th element

Combine

Adapter	Wrapper Type	Description
<code>.chain(other)</code>	<code>Chain<I, J></code>	Append another iterator after this one
<code>.zip(other)</code>	<code>Zip<I, J></code>	Pair elements from two iterators into tuples
<code>.intersperse(sep)</code>	<code>Intersperse<I></code>	Insert a copy of <code>sep</code> between adjacent elements
<code>.intersperse_with(f)</code>	<code>IntersperseWith<I, F></code>	Insert element from closure between adjacent elements

Peek / Inspect

Adapter	Wrapper Type	Description
<code>.peekable()</code>	<code>Peekable<I></code>	Allow peeking at next element without consuming it
<code>.inspect(f)</code>	<code>Inspect<I, F></code>	Call closure <code>f</code> with a reference to each element (for debugging)

Loop / Reverse

Adapter	Wrapper Type	Description
<code>.cycle()</code>	<code>Cycle<I></code>	Repeat the iterator endlessly (requires <code>Clone</code>)
<code>.rev()</code>	<code>Rev<I></code>	Reverse direction (requires <code>DoubleEndedIterator</code>)

Indexing

Adapter	Wrapper Type	Description
<code>.enumerate()</code>	<code>Enumerate<I></code>	Yield <code>(index, element)</code> tuples

Clone / Copy

Adapter	Wrapper Type	Description
<code>.copied()</code>	<code>Copied<I></code>	Copy each element (<code>&T → T</code> where <code>T: Copy</code>)
<code>.cloned()</code>	<code>Cloned<I></code>	Clone each element (<code>&T → T</code> where <code>T: Clone</code>)

Stateful

Adapter	Wrapper Type	Description
<code>.scan(init, f)</code>	<code>Scan<I, F></code>	Stateful fold that yields intermediate values
<code>.fuse()</code>	<code>Fuse<I></code>	Ensure iterator continues to return <code>None</code> after first <code>None</code>

Borrow

Adapter	Wrapper Type	Description
<code>.by_ref()</code>	<code>&mut I</code>	Borrow the iterator instead of consuming it

Example Exercises

1. `map` — Capitalize words

```
fn capitalize_words(words: &[&str]) -> Vec<String> {
    words.iter().map(|w| w.to_uppercase()).collect()
}

fn main() {
    let result = capitalize_words(&["hello", "world"]);
    assert_eq!(result, vec!["HELLO", "WORLD"]);
}
```

2. `filter` — Keep even numbers

```
fn evens(nums: &[i32]) -> Vec<i32> {
    nums.iter().filter(|&&x| x % 2 == 0).copied().collect()
}

fn main() {
    assert_eq!(evens(&[1, 2, 3, 4, 5, 6]), vec![2, 4, 6]);
}
```

3. `filter_map` — Parse strings, discard failures

```
fn parse_numbers(strings: &[&str]) -> Vec<i32> {
    strings.iter().filter_map(|s| s.parse::<i32>().ok()).collect()
}
```

```

}

fn main() {
    assert_eq!(parse_numbers(&["1", "two", "3", "four"]), vec![1, 3]);
}

```

4. `flat_map` — Expand each element into sub-iterators

```

fn repeat_each(nums: &[i32], times: usize) -> Vec<i32> {
    nums.iter()
        .flat_map(|&n| std::iter::repeat(n).take(times))
        .collect()
}

fn main() {
    assert_eq!(repeat_each(&[1, 2, 3], 2), vec![1, 1, 2, 2, 3, 3]);
}

```

5. `flatten` — Flatten nested collections

```

fn flatten_matrix(matrix: Vec<Vec<i32>>>) -> Vec<i32> {
    matrix.into_iter().flatten().collect()
}

fn main() {
    let matrix = vec![vec![1, 2], vec![3, 4, 5], vec![6]];
    assert_eq!(flatten_matrix(matrix), vec![1, 2, 3, 4, 5, 6]);
}

```

6. `zip` — Combine two iterators

```

fn dot_product(a: &[i32], b: &[i32]) -> i32 {
    a.iter().zip(b.iter()).map(|(x, y)| x * y).sum()
}

fn main() {
    assert_eq!(dot_product(&[1, 2, 3], &[4, 5, 6]), 32); // 1*4 + 2*5 + 3*6
}

```

7. `chain` — Append iterators

```
fn merge_sorted(a: Vec<i32>, b: Vec<i32>) -> Vec<i32> {
    a.into_iter().chain(b).collect()
}

fn main() {
    assert_eq!(merge_sorted(vec![1, 2], vec![3, 4]), vec![1, 2, 3, 4]);
}
```

8. `enumerate` — Track index while iterating

```
fn print_with_index(items: &[&str]) {
    for (i, item) in items.iter().enumerate() {
        println!("{}", i, item);
    }
}
```

9. `skip` + `take` — Pagination

```
fn paginate<T: Clone>(items: &[T], page: usize, per_page: usize) -> Vec<T> {
    items
        .iter()
        .skip(page * per_page)
        .take(per_page)
        .cloned()
        .collect()
}

fn main() {
    let data = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
    assert_eq!(paginate(&data, 0, 3), vec![1, 2, 3]);
    assert_eq!(paginate(&data, 1, 3), vec![4, 5, 6]);
}
```

10. `take_while` — Take while condition holds

```
fn take_until_negative(nums: &[i32]) -> Vec<i32> {
    nums.iter().take_while(|&x| x >= 0).copied().collect()
}

fn main() {
```

```
    assert_eq!(take_until_negative(&[2, 4, -1, 3]), vec![2, 4]);  
}
```

11. `skip_while` — Skip while condition holds

```
fn skip_leading_zeros(nums: &[i32]) -> Vec<i32> {  
    nums.iter().skip_while(|&x| x == 0).copied().collect()  
}  
  
fn main() {  
    assert_eq!(skip_leading_zeros(&[0, 0, 1, 2, 3]), vec![1, 2, 3]);  
}
```

12. `step_by` — Take every nth element

```
fn every_other<T: Clone>(items: &[T]) -> Vec<T> {  
    items.iter().step_by(2).cloned().collect()  
}  
  
fn main() {  
    assert_eq!(every_other(&[1, 2, 3, 4, 5]), vec![1, 3, 5]);  
}
```

13. `peekable` — Look ahead without consuming

```
fn compress_runs(nums: &[i32]) -> Vec<(i32, usize)> {  
    let mut iter = nums.iter().peekable();  
    let mut result = Vec::new();  
    while let Some(&n) = iter.next() {  
        let mut count = 1;  
        while iter.peek() == Some(&n) {  
            iter.next();  
            count += 1;  
        }  
        result.push((n, count));  
    }  
    result  
}  
  
fn main() {  
    assert_eq!(  
        compress_runs(&[1, 1, 2, 3, 3, 3]),  
        vec![(1, 2), (2, 1), (3, 3)]  
    );  
}
```

```
    );  
}
```

14. `scan` — Running total (stateful fold)

```
fn running_total(nums: &[i32]) -> Vec<i32> {  
    nums.iter()  
        .scan(0, |acc, &x| {  
            *acc += x;  
            Some(*acc)  
        })  
        .collect()  
}  
  
fn main() {  
    assert_eq!(running_total(&[1, 2, 3, 4]), vec![1, 3, 6, 10]);  
}
```

15. `cycle` — Repeat infinitely

```
fn traffic_light(cycles: usize) -> Vec<'static str> {  
    ["green", "yellow", "red"]  
        .iter()  
        .cycle()  
        .take(cycles)  
        .copied()  
        .collect()  
}  
  
fn main() {  
    assert_eq!(traffic_light(5), vec!["green", "yellow", "red", "green", "yellow"]);  
}
```

16. `rev` — Reverse

```
fn reverse_string(s: &str) -> String {  
    s.chars().rev().collect()  
}  
  
fn main() {  
    assert_eq!(reverse_string("hello"), "olleh");  
}
```

17. `inspect` — Debug during chaining

```
fn double_evens(nums: &[i32]) -> Vec<i32> {
    nums.iter()
        .inspect(|x| println!("before filter: {}", x))
        .filter(|&&x| x % 2 == 0)
        .inspect(|x| println!("after filter: {}", x))
        .map(|x| x * 2)
        .inspect(|x| println!("after map: {}", x))
        .collect()
}
```

18. `copied` / `cloned` — Dereference references

```
fn sum_of_refs(nums: &[i32]) -> i32 {
    nums.iter().copied().sum() // &i32 → i32 via Copy
}

fn sum_of_string_lens(strings: &[String]) -> usize {
    strings.iter().cloned().map(|s| s.len()).sum() // &String → String via Clone
}
```

19. `intersperse` — Insert separator

```
fn join_with_comma(words: &[&str]) -> String {
    words.iter().copied().intersperse(", ").collect()
}

fn main() {
    assert_eq!(join_with_comma(&["a", "b", "c"]), "a, b, c");
}
```

Consuming Adapters (trigger lazy evaluation)

These are **not** wrappers — they consume the iterator and produce a value:

Consumer	Description
<code>.collect()</code>	Collect into a collection (Vec, HashMap, String, etc.)
<code>.sum()</code>	Sum all elements

Consumer	Description
<code>.product()</code>	Multiply all elements
<code>.count()</code>	Count elements
<code>.fold(init, f)</code>	Reduce to single value with accumulator
<code>.reduce(f)</code>	Like fold but uses first element as initial value
<code>.for_each(f)</code>	Call closure on each element
<code>.all(p)</code> / <code>.any(p)</code>	Test predicate against all/any elements
<code>.find(p)</code>	Find first element matching predicate
<code>.position(p)</code>	Find index of first element matching predicate
<code>.max()</code> / <code>.min()</code>	Maximum/minimum element
<code>.last()</code>	Last element
<code>.nth(n)</code>	n-th element (0-indexed)

Rustlings Exercises That Cover Iterators

Exercise	Adapters Used	File
vecs2	<code>.map()</code>	<code>exercises/05_vecs/vecs2.rs</code>
iterators1	<code>basics (.next())</code>	<code>exercises/18_iterators/iterators1.rs</code>
iterators2	<code>.map()</code>	<code>exercises/18_iterators/iterators2.rs</code>
iterators3	<code>.map()</code>	<code>exercises/18_iterators/iterators3.rs</code>
iterators4	<code>.fold()</code> / <code>.product()</code>	<code>exercises/18_iterators/iterators4.rs</code>
iterators5	<code>.filter()</code> , <code>.flat_map()</code> / <code>.flatten()</code>	<code>exercises/18_iterators/iterators5.rs</code>

Not covered by rustlings (write your own!)

`filter_map`, `zip`, `chain`, `peekable`, `rev`, `skip`, `take`, `take_while`, `skip_while`, `step_by`, `inspect`, `scan`, `cycle`, `fused`, `copied`, `cloned`, `intersperse`, `map_while`